Rivet — Project Profile

Rivet — Project Profile

Track 1: Development of Multimodal Content Creation Tools Radeon PRO W7900 · 48 GB · ROCm 7.2.1

The problem

A brand can generate a thousand advertisements with a diffusion model in an afternoon, and ship none of them.

The reason is not quality. It is that a generative model redraws whatever it is given. It bends the logo, invents a fourth button on the product, and renders text as plausible-looking glyphs that spell nothing. A marketing team cannot put that in front of customers, and a regulated brand cannot put it anywhere at all. So the output goes to a designer to be rebuilt by hand, and the generative step has saved nobody any time.

The industry response has been to ask for trust: better prompts, better models, human review at the end. Review does not scale to a thousand assets, and trust is not evidence.

What Rivet does

Rivet takes a product photograph, a brand kit and a short brief, and produces a three-scene vertical advertisement with narration and captions — on one Radeon GPU, with no network access.

It is built on two commitments.

1. Protected layers never pass through a generative model.

The model generates the background, and nothing else. The product cutout, the logo and every character of text are composited afterwards, deterministically, from the original files. This is structural, not a guideline: the compositing stage reads the source assets directly, and the audit verifies that what landed in the frame is pixel-identical to what came in.

2. Evidence before claims.

Every export carries a Campaign Receipt: the sha256 of each input, the seeds, per-stage wall time and peak VRAM, the model revisions, every audit check with its observed value, and any repair that was applied. The pack ships with a manifest hashing every file in it. A receipt is not a summary written after the fact — it is the record the pipeline produced while running, and the numbers in this document come from it.

Who it is for

User	Scenario
A brand marketing team	Needs fifty product advertisements this quarter and cannot send each one through legal review. Rivet blocks the ones that violate the brand’s claim policy before a human ever sees them.
An agency	Delivers to a client whose contract specifies logo treatment and required legal text. The receipt is the proof of compliance, per asset.
A regulated seller	Cannot say “clinically proven” or “guaranteed”. Forbidden claims are checked in headline, support, call-to-action **and spoken narration**, and repaired automatically.
A solo seller	Has one product photo and no design tools. The deterministic compositor gives typography that is correct rather than merely generated.

Verification

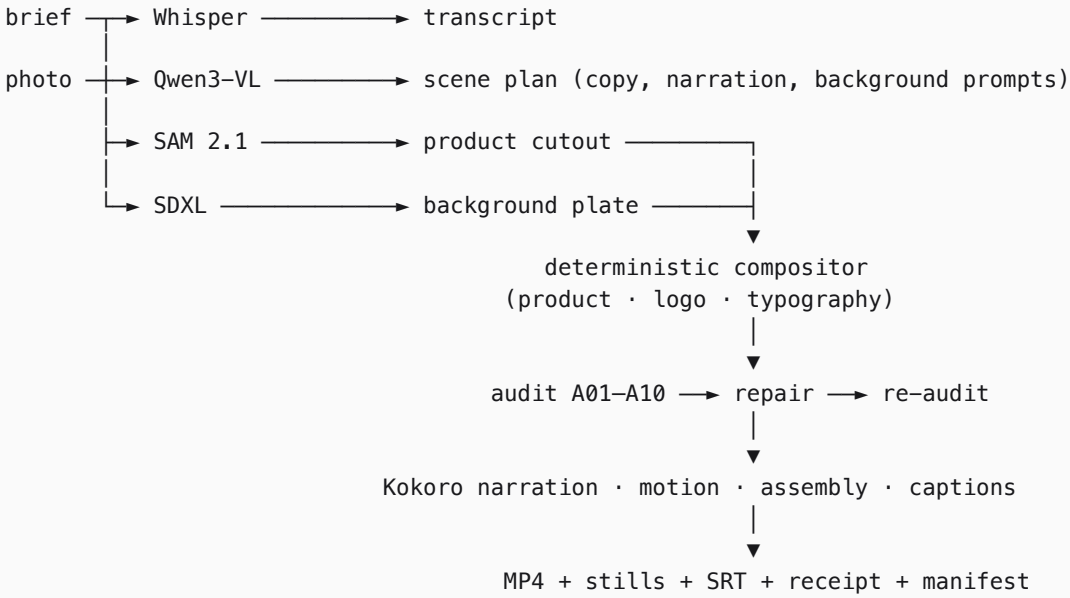
Ten checks run on every scene. Nine are deterministic and block the export; one is advisory.

Check	Verifies	Method
A01	product and logo match the brand-confirmed assets	sha256 of the registered asset against the bytes actually used
A02	the logo in the frame is the real logo	pixel diff against the source, placed by the same geometry the compositor used
A03	rendered copy equals approved copy	exact string comparison
A04	the background stays on-brand	hue distance of every significant colour cluster against the palette
A05	text sits inside its safe area	geometry plus a re-run of the compositor's fit on the actual copy
A06	the product is prominent enough	share of frame from the cutout alpha and the placement transform
A07	no forbidden claims, all required phrases	whole-word matching across copy **and narration**
A09	the product in the frame is the real product	pixel diff against the cutout at its placed position
A10	the text is legible	WCAG contrast against the measured background behind each text box
A08	semantic fit (advisory, never blocks)	Qwen3-VL scores the rendered scene against purpose and audience

When a claims check fails, Rivet does not simply refuse. It rewrites the offending copy, recomposites the scene, re-audits it and records a repair entry stating what changed and whether the result passed. The audit runs **before** the video is rendered, so the exported film is built from the repaired frames rather than certified separately from them.

If a deterministic check still fails, no pack is written and the project moves to `needs_repair`. A blocked export is the product working.

Architecture



Every heavy model sits behind a `Stage` protocol with an explicit resource estimate. Stages are content-addressed: a cache key covers the stage version, model revision, dtype, seed, canonical config and the hash of every input artifact, so a rerun reuses only work whose inputs are genuinely unchanged.

Component	Choice
Planning and semantic audit	Qwen3-VL-4B-Instruct
Backgrounds	SDXL 1.0 with the fp16-fix VAE
Segmentation	SAM 2.1 Hiera Small
Narration	Kokoro-82M
Transcription	Whisper
Compositing, audit, receipt	deterministic Python — no model involved
Runtime	PyTorch on ROCm; FastAPI, SQLite, FFmpeg

Adapting to the Radeon PRO W7900

One model resident at a time. The pipeline runs five models but never holds two. Heavy models are acquired through a residency scheduler: requesting a different model releases the previous one and empties the allocator cache first. This makes the VRAM headroom rule structural rather than a convention someone has to remember, and it removes repeated load cost — SDXL previously rebuilt its pipeline and VAE from disk for every scene.

Measured, not asserted. `rivet benchmark --mode residency` runs the pipeline twice, once with residency enabled and once with it disabled, and writes both to one report. The optimisation is reproducible by a command a judge runs, not a number quoted from a private session.

Residency trades memory held for time saved, which is the right trade on 48 GB of VRAM and the wrong one on a small machine: holding SDXL between scenes on a 16 GB laptop pushes it into swap and the run slows to a crawl. `RIVET_MODEL_RESIDENCY=0` disables it, and the flag is what the benchmark's comparison pass sets.

Honest instrumentation. Peak VRAM is reported only where a true per-stage peak exists — `torch.cuda.max_memory_allocated`, reset before each stage, which on ROCm covers the Radeon path. Where no such counter exists the report prints `n/a` and states why, rather than printing a process-wide figure that would look like evidence. Every report carries the metric it used.

Deterministic by construction. Seeds are threaded explicitly and generators are created per call, so reusing a resident pipeline cannot change an output. The benchmark verifies this empirically: each run is digested over the rendered still bytes, the seeds and the deterministic audit observations, and the digests are compared across runs. The comparison cannot be satisfied by matching paths or identifiers, only by matching content.

Offline by requirement. Models resolve from local snapshots. `make offline-demo` runs the golden project with every outbound socket blocked at the Python socket layer; the run completes with zero connection attempts.

Measured results

Radeon PRO W7900 (gfx1100, 49136 MB) · ROCm 7.2.53211 · PyTorch 2.9.1 · fixture `kora-arc`. Every figure below comes from `rivet benchmark` and is reproduced by the commands in the next section. The full reports are in `docs/benchmarks/`.

	seconds	audit
Cold: plan, generate, audit, render, pack	**67.8**	27/27
Hot: same work, models already resident	**41.2**	27/27
Offline gate, every outbound socket blocked	67	27/27

Peak VRAM is **9168 MB of 49136** — the pipeline holds one heavy model at a time and leaves 81% of the card free, which is why a larger diffusion model would fit without changing the design.

The residency optimisation

SDXL rebuilt its pipeline and VAE from disk for every scene, and Kokoro and Qwen3-VL reloaded per call. Heavy models now pass through a residency scheduler: acquiring a different model releases the previous one first, so exactly one is resident and the VRAM headroom rule holds by construction.

arm	runs	median total
resident	2	**42.2s**
reload per stage	2	55.8s

13.7 seconds faster, a 24% reduction end to end and 32% on the generation phase alone. The two arms alternate after a discarded warmup, because a single resident-then-reload pair would hand the second arm a warm page cache — worth about 17s on this pipeline, more than the effect being measured. The first, unalternated attempt reported residency as 4.4s *slower*; that result was measuring run order and was discarded rather than published.

Determinism

The same content digest, `347606b58a93ba16`, appears in all eleven runs above — cold, hot, warmup, and both residency arms. The digest covers the rendered still bytes, the seeds and the deterministic audit observations, so it cannot be satisfied by matching paths or identifiers, only by identical output. Reusing a resident pipeline does not change a single pixel.

Platform notes

- SDXL attention runs through ROCm's **AOTriton** efficient-attention path on gfx1100, not a generic fallback.
- ROCm's tuning guidance recommends disabling NUMA auto-balancing; the container has no host-level privileges to do so, so the figures above are measured with it enabled and would if anything improve.

Reproducing the results

```
uv sync --extra gpu
make doctor
make offline-demo                                  # gate G2: no network, passing export
make test-golden                                   # byte-identical compositing
uv run rivet benchmark --fixture fixtures/kora-arc --mode residency
```

Every figure published for this submission is produced by these commands on the supplied W7900. Figures measured on a development machine are recorded as development numbers and are never quoted as results.